미니 React 동작 복습

Week11-12 · AI 활용 학습 퀴즈 정리 · 11/11 갱신

현재 실습은 **5×5 불 끄기 퍼즐**이다. 기존 미니 React 엔진을 사용하며, 댓글 화면에서 학습한 내용은 이전 시연 예시로 남겼다.

1. 실제 영역과 테스트 영역

첫 로드에서는 실제 영역에서 만든 **첫 VDOM을 미리보기에도 재사용**한다. HTML을 편집하면 미리보기부터 갱신되고, **Patch 적용**을 눌러야 실제 영역이 바뀐다.

퍼즐 화면 하단의 **VDOM 실험실**(`/lab.html`)에서 위 흐름을 직접 확인할 수 있다.

버튼 클릭과 부모 이벤트 전파

아래 HTML을 입력하고 **Patch 적용**을 누른 뒤 실제 영역에서 버튼을 클릭한다. 첫 버튼은 `1234`를 띄운다. 내부 버튼의 클릭은 부모 `div`로 전파되어 두 번째 팝업이 뜬다.

```
<button onclick="alert('1234')">aaa</button>
<div onclick="alert('부모 div 클릭')">
  <button>내부 버튼</button>
</div>
```

`src/lab.js`는 `onclick` 코드를 VDOM에 문자열로 보관하고, 실제 영역에서 해당 요소의 클릭 핸들러로 연결한다. `this`와 `event.currentTarget`은 핸들러가 연결된 요소를 가리키며 `event.stopPropagation()`은 부모로의 전파를 막는다. 미리보기는 클릭 코드를 실행하지 않는다.

2. Diff와 Patch

Diff는 이전 Virtual DOM과 새 Virtual DOM을 비교해 변경의 위치와 종류를 찾는다. **Patch**는 그 변경을 실제 DOM에 적용한다. `<h3>`의 글자만 달라지면 요소를 재사용하고 그 안의 텍스트만 바꾼다.

실험실은 클릭 코드가 DOM을 수정한 경우에도 Patch 시점의 실제 DOM을 읽어 목표 VDOM과 비교한다. 버튼 글자만 수정하는 테스트에서는 기존 버튼·부모 요소·이벤트 핸들러가 유지되고, `MutationObserver`에 텍스트 변경 1건만 기록됐다.

퍼즐도 같은 엔진을 사용한다. 칸을 누르면 `setGame()` → `FunctionComponent.update()` → 새 VDOM 생성 → `diff()` → `patch()` 순서로 진행된다. 중앙 클릭 테스트에서는 기존 버튼 25개를 재사용하고 5개 칸의 켜짐 상태가 바뀌는지 확인한다. 엔진이 두 함수를 호출하는 곳은 `src/component.js`, 함수 구현은 `src/vdom.js`에 있다.

3. State History

`A → B → C`에서 뒤로가기를 누르면 기록의 현재 위치만 `B`로 이동한다. 실제 영역과 미리보기, HTML 편집기가 함께 복원된다. 여기서 `D`를 적용하면 기록은 `A → B → D`가 되고, 앞으로가기로 `C`에 갈 수 없다.

퍼즐의 **되돌리기 / 다시 실행**도 이동 기록의 현재 위치를 바꾼다. 되돌린 뒤 다른 칸을 누르면 앞쪽 기록을 교체한다. **이 퍼즐 처음부터**는 시작 보드를 유지하며 이동 이력을 비운다.

실험실 이력은 `onclick` 코드와 HTML을 함께 복원한다. 편집창에서도 `onclick`을 다시 수정하거나 삭제할 수 있다.

4. 루트 상태와 Props

퍼즐에서는 루트 `Puzzle`이 `useState`로 게임 기록과 미리보기 위치를 관리한다. `LightGrid`와 `GameControls`는 루트에서 받은 props로 화면을 만든다. 퍼즐 쪽 Hook 호출은 `src/game.js`에서 확인할 수 있다.

이전 댓글 시연

댓글 초안은 루트 컴포넌트 `Board`가 `useState`로 기억한다. 작성창을 닫으면 기존 `textarea`는 DOM에서 제거된다. 다시 열면 `CommentComposer`가 `draft`를 props로 받아 새 `textarea`의 `value`에 넣는다. 초안이 비면 안내 문구가 보인다.

댓글 옆은 **등록 시·분 → 답글 → 좋아요** 순서다. 답글은 원래 댓글 아래에 표시되며, `답글 펼치기 ↓ / 접기 ↑`로 목록을 전환한다. 새 답글을 등록하면 목록이 펼쳐진다.

- **등록** 후 `Board`가 상태에 댓글을 추가하면 `CommentList`가 props로 받아 표시하고, `useEffect`가 저장한다.
- **새로고침** 후 `Board`가 `storage.read("comments")`로 같은 서버 ID의 값을 복원하면 `CommentList`가 props로 받아 표시한다.

5. useMemo와 useEffect

- `useMemo`는 게임 상태를 바탕으로 보드와 남은 불을 계산한다. 미리보기 위치만 바뀌면 보드 계산 결과를 재사용한다.
- `useEffect`는 DOM 반영 뒤 게임 기록을 저장하고 남은 불 또는 성공 상태를 탭 제목에 표시한다.
- `src/component.js`의 `claim()`은 세 Hook(`useState`, `useMemo`, `useEffect`)이 호출 순서에 맞는 저장 자리를 찾게 한다.

이전 댓글 시연

- **`useMemo`**: `comments`가 같으면 요약과 정렬 결과를 재사용한다. 댓글 좋아요가 바뀌면 내림차순으로 정렬하고, 동점은 등록순이다. **1위가 10개 이상**이면 인기 댓글을 표시한다.
- **`useEffect`**: DOM에 변경을 반영한 뒤 실행된다. 댓글 시연에서는 좋아요 수를 탭 제목에 반영하고, 좋아요와 댓글 상태를 브라우저에 저장했다.

6. 새로고침과 서버 재시작

`localStorage`에 **서버 실행 ID와 값**을 함께 저장한다. 같은 브라우저에서 새로고침하면 퍼즐의 시작 보드, 이동 기록, 현재 위치와 실험실의 Patch 기록이 복원된다. 서버를 재시작하면 새 ID를 받아 새 퍼즐과 처음 HTML 샘플로 시작한다.

이전 댓글 시연도 같은 방식으로 좋아요·댓글·대댓글·접기 상태·초안을 복원했다. 서버가 재시작되면 좋아요 0과 빈 댓글에서 시작했다.

7. 5×5 불 끄기 퍼즐

- 칸을 누르면 그 칸과 상하좌우의 불이 뒤집힌다. 중앙은 5칸, 가장자리는 4칸, 모서리는 3칸이 바뀐다. 모든 불을 끄면 성공이며, 이후에는 되돌리거나 새 퍼즐을 시작할 수 있다.
- 꺼진 보드에서 합법적인 클릭으로 시작 보드를 만들기 때문에 해가 있다. 영향을 받는 칸을 미리 볼 수 있고, 방향키와 Enter·Space로도 조작할 수 있다.
- `npm run dev` 후 `http://127.0.0.1:4173`에서 실행한다. `npm test`로 **브라우저 테스트 30개**를 통과했다. 퍼즐 생성 100건, 이력·저장 복원과 함께 실험실 클릭 전파, 핸들러 변경·삭제, DOM 부분 갱신을 확인했다.

불이 하나 남았을 때

왼쪽 위를 1행 1열로 센다. 마지막 불의 위치에 맞춰 각 행에 적힌 열을 한 번씩 누른다. 순서는 상관없으며, 풀이 중 다른 불이 다시 켜질 수 있다.

남은 불 (행, 열)	1행	2행	3행	4행	5행
(2, 2)	3	2, 3, 4	1, 2, 5	2, 5	3, 4
(2, 4)	3	2, 3, 4	1, 4, 5	1, 4	2, 3
(3, 3)	1, 2	3	1, 3, 4	1, 5	2, 3, 5
(4, 2)	2	1, 2, 3	4	1, 2, 4, 5	2, 4
(4, 4)	2, 3	1, 4	1, 4, 5	2, 3, 4	3

관련 노트 · `react-study-notes.md`